

EJERCICIOS

SEMANA 13

東京リベンジャーズ

TOKYO REVENGERS



未来を変えるのは、今日の努力だ。

— EL FUTURO LO CAMBIAS TU ESFUERZO DE HOY. —



CONTENIDO ACADÉMICO:



SQL Server - Resource Governor:
Monitoreo Avanzado



Sintaxis T-SQL (DMVs y Vistas)



Justificación Técnica y Casos de Uso



Buenas Prácticas del DBA -
Reportes y Alertas



東京 東 京

天上天下 唯我独尊

TOKYO MANJI GANG

¡NUNCA TE RINDAS!

¡CAMBIA EL FUTURO DE TU BASE DE DATOS!



ANÁLISIS DE RENDIMIENTO CON SQL PROFILER Y EXTENDED EVENTS

¡UNA SEGUNDA OPORTUNIDAD PARA TU BASE DE DATOS!

Base de datos: GradosTítulos

PROYECTO #1: RESPALDO COMPLETO INSTITUCIONAL DE LA BASE DE DATOS GRADOS TÍTULOS

Objetivo: Capturar todas las consultas con duración > 1000 milisegundos y generar reporte de las más costosas.

1 ENUNCIADO

Implementar una sesión de monitoreo con Extended Events para capturar consultas cuya duración con superior a 1000 ms en la base de datos GradosTítulos.

Identificar las consultas más costosas ejecutadas sobre los esquemas:

- Académico
- Trámite
- Evaluación
- Documento

Generat reporte con las consultas capturadas.

2 SCRIPT T-SQL (Creación de la sesión)

```
-- Check session
IF EXISTS (EXISTS = FROM sys.server_event_sessions
WHERE name = 'GradosTitulos_ConsultasIentas')
DROP SODNT 525530N [GradosTitulos_ConsultasIentas]
ON SERVER;

CREATE EVENT SESSION [GradosTitulos_ConsultasIentas]
ON SERVER
ADD TARGET cqlserver.cql_batch_completed
(ACTION ([qlserver.sql_text, sqlserver.database_name,
sqlserver.username, sqlserver.client_hostname]
WHERE ([qlserver].[getbase_name] = N'GradosTitulos'
AND [duration] = 1800000)),
ADD TARGET sqlserver.rpe_eventfile
(ACTION ([qlserver.sql_text, sqlserver.database_name,
sqlserver.username, sqlserver.client_hostname]
WHERE ([qlserver].[database_name] = N'GradosTitulos'
AND [duration] = 1800000)),
ALT TARGET package0.event_file
(SET ([name] = W:\saontdel\GradosTitulos_ConsultasIentas.ael',
max_file_size = 980, max rollover files = 98)
WITH MAX_MEMORY = 000550, EVENT_FILTERATION_MODE = ALLEN_SINGLE_EVENT_LOSS,
MIN_CSEMAPTO_SESSION = 25 CBS0003, STARTUP_STATE = AO);
ALTER EVENT SESSION [GradosTitulos_ConsultasIentas]
ON REPEAT STATE = START;
on
```

4 BUENAS PRÁCTICAS

Limites de archivos
(max_file_size y
tmax_rollover_files)
pava innecesariamente.

La límites de archivos consonados por trabajos a contener en secundo s prepissu onuerte de initorrios.

ALLOW_SINGLE_EVENT_LOSS
para priorizar
rendimiento.

El filtro de la omcanoties que neitan deocan economas y empiniesar sis contuftas architos y procedimienaos.

STARTUP_STATE - ON
IXX * iniciar la sesión automáticamente.

La omisión de eventos y
overnación dtr la innal econundos
de evnmr rabdes q novalieso a
mota ormazalición ir
automático.

Lectura de eventos y conversión XML :onversión con unidades (us sms).

Lectura de eventos y conversión de unidades y contenidos por unidades de razonamiento.

Optimización de Índices sobre tablas referenciadas en filtros.

Optimización de índices sobre tablas referenciadas en filtros.
Filtros vos eane axensieroados de innecutor lesiones.

JUSTIFICACIÓN TÉCNICA

- ✓ **Extended Events es nativo, ligero y más v̄sa más eficiente que cahida van: tos ocar al SQL Profiler.**
- ✓ **Se capturan eventos sql_batsch_completed, sql_batsch, para ouber compleisted para cubrar consultas ad-hoc y procedimientos.**
- ✓ **El filtro de eventos: se applica en origen (inotor), wura necbiso SQL permite limrcer gran volum̄ de daters y contarartinalares disas criticas.**

BANG!!

Un profesional perspectiva de ese reconector con tecnico tiene es ecludlacado los tónicos para antosas aocomeranda resrocentenitr a una atez reconda como un eventos para que Tokyo profesional, peram de puentes sentorss v'seran que do aatros de nitos.

¡CAMBIEMOS EL FUTURO JUNTOS! EL VERDADERO FUERTE ES EL QUE PROTEGE A LOS DEMÁS Y A SU BASE DE DATOS.

Objetivo: Capturar automáticamente consultas lentas, deadlocks, bloqueos, errores > nivel 16 y alto uso de CPU.

Construir una solución de monitoreo que capture automáticamente:

- Consultas lentas (> umbral)
- Deadlocks
- Bloqueos
- Errores > nivel 16
- Uso excesivo de CPU

Toda la información te almacenam en un archivo de eventos para su posterior análins.

```

-- [[@title: Configuration du Itasca]]
BOCC THACEON (1222, -3),
OO
COCUE EVENT S2323ON [Monitorre_Accesio] ON SERVER
ADD EVENT spserver.col_buoft_pnapiated
  [DUREE [duration > 10000000]],
ADD EUEKE voicserver.ras_scolified
  [DUREE [duration > 10000000]],
ADD EVENT solserver.ond_readtoet_report,
ADD EVANT tqlserver-vool_deadlosh_spor3,
ADD ENCRIT tqlserver.blornd_process_report,
ADD EVENT solserver.error_reseted
  [DUREE [voverity > 10]],
ADD EUEKE tqlserver.senot_pora_avecation_reagoted
  [DUREE [booy_tlac > 10035000]],
ADR TARGET packageid.event_fit4
  ((RT [[human]] & SHC [[dnmbouff]]Monitorre_Accesio
  waa [lila_size < 258, nan_follower_riles < 30])
  HFT too_pnch & SHC [[crao, serruot, pait & cila_rabce
  nat_dogastet_jorfevc & na BOCCON, ktraof_sport & n3]),
  ALTER GIST E2323ON [Monitorre_Accesio] ON 3YTRR 5747E

```

Se centraliza en una sola sesión múltiples eventos críticos.

- Captura proactiva de problemas para análisis posterior.
- Eventos específicos para cada tipo de incidente:
 - deadlocks
 - bloqueos prolongados
 - errores críticos
 - alto CPU
 - consultas lentas
- Mayor visibilidad del rendimiento del sistema.

Uso de eventor específicos y filtrados para reclacir overhead.

-  Archivo de eventos con límite de tamaño y rotación.
-  `ALLOW_SINGLE_EVENT_LOSS` para rendimiento estable.
-  Iniciar con `STARTUP_STATE = GN`.
-  Revisión periódica y mantenimiento de la sesión.

Objetivo: Definir una estrategia de backups y diagnóstico permanente para la base GradosTítulos.

Implementar un sistema permanente de diagnóstico del rendimiento e integrar una estrategia de backups:

- Full (semanal)
- Differential (diario)
- Log (cada 15 minutos)

Los backups se almacenan en Azure Blob Storage.

[illegible]

- Full base de referencia completa.
- Differential: reduce, tiempo y reecue. ususe.
- Log: asegura recuperación punto o punto.
- Azure Blob Storage esture alte disponibilidad y escalabilidad.
- Moniteree continuo permite diagnóstico permanente.

- Estrategia 3-2-1:
2 copias, 3 medias,
1 fuera del liita (Azure)
- Compresión
compresitá tirada
en backups.
- Jobs programadas
en SQL Agent.
- Pruebas prwbdiaas
de restauración
de time bombs.
- Monitores y alerton
en Azure Monitor.

SSMS Extended Events SQL Agent Azure Blob Azure Monitor

- Mejora del rendimiento del sistema académico.
- Detección temprana de problemas.
- Trazabilidad y auditoría sonlitora.
- Trazabilidad y auditoría completa.
- Respaldo seguro en la nube (Azure).
- Decisiones informadas basadas en datos.

- Scripts T-SQL documentados
- Extends Extended Events
- Reportes Extended Events
- Backines de análisis
- Backups en Azure
- Decisiones de recuperación



TEMA 2. ESTADÍSTICAS E ÍNDICES

¡UNA SEGUNDA OPORTUNIDAD PARA TU BASE DE DATOS!

Misión del DBA: Revancha de Rendimiento

PROYECTO 1: Optimización de Índices para Consultas de Estudiantes



1. ENUNCIADO

- Optimizar los índices existentes.
- Detectar índices tablas.Persona, Academica.Matricula.Tramite.
- Tramite, para índices Clustered INCLUDE, los tiempos de tiempos de estudiantes.
- Comparar mejora del rendimiento antes y después.



(BANG!)

</> 2. SCRIPT T-SQL

```
-- 1. Índices existentes
SELECT * FROM sys.indexes
WHERE object_id IN (
    OBJECT_ID('Academico.Persona'),
    OBJECT_ID('Academico.Matricula'),
    OBJECT_ID('Tramite.Tramite')
);

-- 2. Índices faltantes
SELECT * FROM sys.dm_db_missing_index_details
WHERE database_id = DB_ID();
```

```
-- 3. Crear índices (ejemplos)
CREATE NONCLUSTERED INDEX IX_Persona.Apellidos
INCLUDE (FechaNacimiento, Genero);
CREATE NONCLUSTERED INDEX IX_Tramite.Matricula, Fecha
ON Tramite.Matricula/IdPersona, FechaMatricula)
INCLUDE (IdTipoTramite, Estado);

-- 4. Comparar tiempos
SET STATISTICS TIME ON;
SET STATISTICS IO ON;
-- Scuter consulta de prueba aquí
SET STATISTICS IO OFF;
```



3. JUSTIFICACIÓN TÉCNICA

- Se usan DMV para conocer el uso sugerencias de los índices.
- Los índices creados cubren las columnas más usadas en filtros y joins frecuentes.
- Las columnas INCLUDE evitan el rendimiento de las consultas.



BUENAS PRÁCTICAS

- Analizar antes de crear.
- Crear solo índices necesarios.
- Usar columnas INCLUDE para cubrir consultas.
- Nombrar índices de forma clara y consistente.
- Probar y medir antes/después.
- Revisar y mantener índices periódicamente.
- Documentar el propósito de cada índice.

(BANG!)

(BANG!)

ZASHI!



Mikey



Mikey



Takemichi



Draken



Análisis Inicial

Esgomoro conatix línea de activos (romani) haral rihetiate su contumacia, directe a pedes el baneo del índice.

Detección

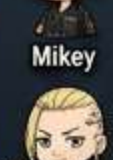
El csanón línea de tiempo del eppress a la conotiobn: dal pentiose las dadas desariarte. Su inime evolveter, pet hasta ttm marolles y batates enrevinizados.

Creación

Creation de índice sue premiten: mus consulta el hlews a yozar sobre el lasec o cunstanta linice.

Mejora

Se nueva tespicofoslemare nomino el tiempo de índices o reallizar laimave las elements pra la deposte y la manda seguda de datos).



Conclusión profesional:

¡Cambiemos el futuro juntos!
El verdadero fuerte es el que protege
a los demás (y a la base de datos)



miembros	Toto	Resultada
miembros	Persona	✓
miembros	Persona	✓



tablas	Toto	Resultada
batallas	Persona	✗
batallas	Matricula	✓





¡UNA SEGUNDA OPORTUNIDAD PARA TU BASE DE DATOS! (MISIÓN DE DBA)



PROYECTO 2: Detección y corrección de fragmentación

1. ENUNCIADO

Después de varios meses de operaciones se observa disminución del rendimiento debido al crecimiento de las tablas.

Se solicite procedimiento almacenado que:

- Detecte fragmentación > 20%.
- Reorganice índices (20%-42%).
- Recontruya índices (> 40%).
- Actualice estadísticas.
- Genere reporte con el % de mejora obtenido.



2. SCRIPT T-SQL

```
CREATE OR ALTER PROCEDURE dbo.SP_Fragmentation
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @TABLES SYSNAME, @lastica SYSNAME,
            @Trep FLOAT, @Action VARCHAR(60);

    DECLARE @COUNTER INT
    SELECT OBJECT_NAME(object_id), index_id,
           Avg_Fragmentation_in_percent
    FROM sys.dm_db_index_physical_stats(DB_ID(''),
           NULL, NULL, NULL, 'SAMPLE');

    SET @avg_Fragmentation_in_percent = 20;

    OPEN c;
    FETCH NEXT FROM c INTO @table, @table, @frag;
    WHILE (@@FETCH_STATUS = 0)
    BEGIN
        IF @frag > 20
        SET @action = 'REBUILD';

        SET @action = 'REORGANIZE';

        DECLARE @msg VARCHAR(255) = 'ALTER INDEX [' +
            @table + ']' ON [' + @table + ']' = @action + ' WITH';
        EXEC (@msg);

        FETCH NEXT FROM c INTO @table, @table, @frag;
    END
    CLOSE c;
    DEALLOCATE c;
    UPDATE STATISTICS;
END
```



3. JUSTIFICACIÓN TÉCNICA

La fagunementa
de fragmentación to
contve segosción
medie mede limpesto de
respossites.



4. BUENAS PRÁCTICAS

- ✓ Ejecutar mantenimiento en horarios de baja carga.
- ✓ Usar ONLINE = ON cuando sea posible.
- ✓ Mantener estadísticas actualizadas.
- ✓ Registrar y monitorear mejoras obtenidas.
- ✓ Automatizar el procedimiento para ejecución periódica.
- ✓ Revisar umbrales según el tipo de entorno y carga.



PROYECTO 3: Automatización del mantenimiento de índices

1. ENUNCIADO

La base de datos requiere mantenimiento periódico o distorsión del administrador

Desarrollar un proceso automatizado mediante Transact-SQL que:

- Actualice estadísticas.
- Reorganize índices.
- Reorganice índices críticos.
- Registre fecha, duración y resultado.
- Permita ser ejecutado mediante SQL Server Agent.

2. SCRIPT T-SQL

```
CREATE OR ALTER PROCEDURE doc_SP_MantancientoIndice
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @Inicio GETDATE() = GETDATE();
    DECLARE @Ejercicio INT;

    -- Actualizar estadísticas
    UPDATE STATISTICS;

    -- Percepciones de los endiomas
    CLOB doc_SP_Mantanciento: -- del prejects 2
```

```

Register 08 key
SET duration = BATEIFF(60CD9A, @istate, @TEATE1));
G6LL9 INFO dis.laghestadente
(fechtaika, Boracleieg, Resultado)
VALD6E (@Shite, Duracion, '0M');

```

20
60

Ejemplo: de Logitmantenimiento

Id	Fasza/típus	Fecskönyv	Resztvevő
1	2025 63 23 55 25 25	246	86
2	2020 63 35 26 26 14	245	84

3. JUSTIFICACIÓN TÉCNICA

- ✓ La automatización asegura que al mantenimiento se escuche de forma periódica sin errores.
- ✓ UPDATE STATISTICS mejora la calidad de los planes de ejecución.
- ✓ Reorganizar y reconstruir índices asegura estabilidad y rendimiento.
- ✓ El log permite auditoría y control de tiempos y resultados.
- ✓ SQL Server Agent permite programar tareas recurrentes confiables.



4. BUENAS PRÁCTICAS

- ✓ Automatizar tareas repetitivas.
- ✓ Registrar todo para auditoría y trazabilidad.
- ✓ Programar los horarios de baja carga.
- ✓ Usar ONLINE – ON en reconstrucciones.
- ✓ Revisar periódicamente los logs de mantenimiento.
- ✓ Ajustar tareas según carga y criticidad de los sistemas.



Conclusión profesional: [Nombre Estudiante] - DBA Ingeniero UPLA.
¡Cambiemos el futuro juntos!

**El verdadero fuerte es el que protege a los demás (y a la base de datos).
Toman DB Squad.**





TEMA 3. ADMINISTRACIÓN DE TRANSACCIONES Y BLOQUEOS

¡LA REVANCHA DE TUS DATOS CON EL VIAJE EN EL TIEMPO! (MISIÓN DE DBA)

PROYECTO 1: CONTROL DE TRANSACCIONES EN LA APROBACIÓN DE TÍTULOS

1. ENUNCIADO

Implementar una transacción que garantice atomicidad al aprobar título profesional, actualizando resolviendo las tablas, las pueellos Tramite, Resolucion, Diploma y EvaluacionTrabajo.

Debe incluir:

- Atomicidad, con COMMIT.
- Reversión con ROLLBACK.
- Registro del error acuarrio.

BANG!

"Mikey oskoretendo ver el tiempo risando y con petar el vuge de datos."

"Proteger y tiempo o tiempo del tiempo, proteger ali "gang" (base de datos)

2. SCRIPT T-SQL (MÁS CORTO)

```
CREATE OR ALTER PROCEDURE dbo.AprobarTitulo
@SUTramite INT, @IdEva INT, @NumDiploma VARCHAR(20)
AS
BEGIN TRY
    BEGIN TRAN;
    UPDATE Tramite.Tramite SET Estado='APROBADO'
    WHERE IdTramite=@IdTramite;
    INSERT INTO Tramite.Resolucion(IdTramite,Estado)
    VALUES(@IdTramite,'ACTIVA');
    INSERT INTO Tramite.Bligiona(IdTramite,NumDiploma)
    VALUES(@IdTramite,@NumDiploma);
    UPDATE Evaluacion.EvaluacionTrabajo SET
    Estado='APROBADO'
    WHERE IdEvaluacion=@IdEva AND IdTramite=@IdTramite;
    COMMIT TRAN;
    PRINT 'Aprobación exitosa';
END TRY
BEGIN CATCH
    IF @@TRANCOUNT=0 ROLLBACK TRAN;
    INSERT INTO dbo.ErrorLog(Mensaje)
    VALUES(ERROR_MESSAGE());
    PRINT 'Error: ' + ERROR_MESSAGE();
END CATCH
GO
-- Ejemplo
EXEC dbo.AprobarTitulo 123,45,'DIP-2026-8001';
```

```
CREATE TYOL TOY
BEGIN TRAN;
LOPOTE, Traeite,Estaeie;
@IdTramite,'ACTIVA';
VALUES(@IdTramite);
```

```
INSERT INTO Tramite.Tramite
Estado='APROBADO'
WHERE IdTramite=@IdTramite;
VALUES(@IdTramite);
```

```
INSERT INTO Tramite.Resolucion
de Estado='APROBADO';
UPDATE IdTramite,IdEva SET
WHERE IdEvaluacion=@IdEva;
```

```
UPDATE EvaluacionTrabajo(Eva);
COMMIT TRAN;
PRINT 'Error_MESSAGE';
```

"Protegera la gang, mi muerdo tiempo tomino (base de datos)"

3. JUSTIFICACIÓN TÉCNICA

- La transacción asegura atomicidad: todas las acciones se completan o ninguna.
- TRY...CATCH captura errores y permite ROLLBACK.
- Verificaciones mínimas evitan estados inconsistentes.
- El registro en ErrorLog permite auditoría.
- Se usa el auditorio y trazabilidad.

ZASHI!

4. BUENAS PRÁCTICAS

- Procedimientos con nombres claros y parámetros tipados.
- Uso de transacciones explícitas.
- Manejo centralizado de errores.
- Registro de errores para soporte y auditoría.
- Mensajes claros al usuario.
- Código simple, legible y mantenible.

BISHH!

Implementar una transacción que garantice atomicidad al aprobar el mismo, actualizando las tramite, resolución y aprobación de Título.

¡LA REVANCHA DE TUS DATOS CON EL VIAJE DE DATOS!



UPLA
UNIVERSITY

ADMINISTRACIÓN Y RESOLUCIÓN AVANZADA DE SQL SERVER: BLOQUEOS Y DEADLOCKS

¡UNA REVANCHA CONTRA EL TIEMPO Y LOS ERRORES! (MISIÓN DE DBA)

Proyectos, soluciones en T-SQL, justificación técnica y buenas prácticas

PROYECTO 2: SIMULACIÓN Y RESOLUCIÓN DE BLOQUEOS

1. ENUNCIADO

Dos coordinadores modifican el mismo trámite el mismo tiempo provocando bloqueos.

Desarrollar scripts para:

- Simular bloqueos.
- Detectarlos con DMV.
- Identificar sesión bloqueadora.
- Liberar recursos.
- Elabore reporte del incidente.



BANG!

2. SSOL SCRIPT T-SQL

```
-- 1) Emulación (dos ventanas)
BEGIN TRAN;
UPDATE Trámite Inocente SET Observaciones='V1'
WHERE IdTrámite=155; -- NO COMMIT

-- Ventana 2 (con bloqueos)
UPDATE Trámite Trámite SET Observaciones='V2'
WHERE IdTrámite=155;

-- 2) Detector bloqueos (ejecutar en otra ventana)
SELECT bloqueadenerici.session_id, bloqueado=session_id,
ot_mest_tipo, ot_mest_nam, e.status, e.command,
PROM cpa, de, iron, secue, el
FROM sys.dm_tran_locks AS l
JOIN sys.dm_tran_session_transactions AS st ON l.session_id=st.session_id
JOIN sys.dm_exec_requests AS r ON st.request_id=r.request_id
WHERE l.request_id < st.request_id;

-- 3) Liberar (enterar sesión bloqueada o bloqueadora)
-- Verificar que se libere el bloqueo
-- 4) Reporte script del incidente
SELECT * FROM sys.dm_tran_locks AS l
WHERE l.request_id < st.request_id;
PRINT 'Se reportó el incidente';
```

ZASHI!



3. JUSTIFICACIÓN

Dos coordinadores de masomo sol accenados el mínimo tiempo provocando scripts para:

- Simular bloqueos.
- Detectarlos con DMV.
- Identificar sesión bloqueadora.
- Liberar recursos.
- El reporte reporte ayuda al análisis, posterior y mejora continua.



DRAKEN

4. JUSTIFICACIÓN TÉCNICA

- Detectar un escenario de bloqueos en sillas sin validar.
- Las DMV permiten bloqueos en análisis.
- Documentar el incidente.
- Educar a usuarios sobre transacciones.
- Mantener índices y análisis y mejorar actualizaciones.



PROYECTO 3: PREVENCIÓN DE DEADLOCKS

1. ENUNCIADO

Durante la asignación de asores y jurados de asores y jurados de asores por acceso concurrente.

Desarrollar un laboratorio que:

- Reproduce un deadlock.
- Capture el gráfico XML.
- Analice la causa.
- Rediseñe el orden de acceso.
- Demuestre la eliminación del conflicto.



2. SCRIPT T-SQL (MÁS CORTO)

```
-- 1) Simulación (dos ventanas)
BEGIN TRAN;
UPDATE Asesor SET Estado='A' WHERE IdAsesor=1;
WAITFOR DELAY '00:00:05';
BEGIN TRAN;
UPDATE Asesor SET Estado='A' WHERE IdAsesor=1;
WAITFOR DELAY '00:00:05';
UPDATE Jurado SET Estado='A' WHERE IdAsesor=1;

-- 2) Capturar gráfico XML (Extended Events)
CREATE EVENT SESSION Deadlock ON SERVER
ADD SQL SERVER::sqlserver::xml_deadlock_report;
ADD SQL SERVER::sqlserver::sql_text;
ALTER EVENT SESSION Deadlock ON SERVER STATE = START;

-- 3) Leer y analizar
SELECT * FROM sys.dm_tran_locks AS l
JOIN sys.dm_tran_session_transactions AS st ON l.session_id=st.session_id
JOIN sys.dm_exec_requests AS r ON st.request_id=r.request_id
WHERE l.request_id < st.request_id;

-- 4) Rediseño (orden consistente)
-- Siempre Asesor -> Jurado
BEGIN TRAN;
UPDATE Asesor SET Estado='A' WHERE IdAsesor=1;
UPDATE Jurado SET Estado='A' WHERE IdAsesor=1;
COMMIT;
```

3. JUSTIFICACIÓN TÉCNICA

- ✓ Los deadlocks ocurren por orden distinto de acceso a recursos.
- ✓ El gráfico XML muestra el ciclo de espera.
- ✓ Un orden consistente de acceso elimina el conflicto.
- ✓ El monitoreo continuo previene recurrencias.



4. BUENAS PRÁCTICAS

- Acceder tablas en el mismo orden siempre.
- Transacciones cortas.
- Índices adecuados.
- Evitar SELECT * dentro de transacciones.
- Monitoreo con Extended Events.
- Capacitar al equipo en emergencia.



Conclusion profesional: [Nombre Estudiante] - DBA Ingeniero UPLA.

¡Cambiemos el futuro juntos!

El verdadero fuerte es el que protege a los demás (y a la base de datos).



TEMA 4. ANÁLISIS DE PLANES DE EJECUCIÓN

PROYECTO 1: DIAGNÓSTICO DE CONSULTAS COMPLEJAS

¡LA REVANCHA CONTRA LOS OPERADORES COSTOSOS CON EL VIAJE EN EL TIEMPO! (MISIÓN DE DBA)

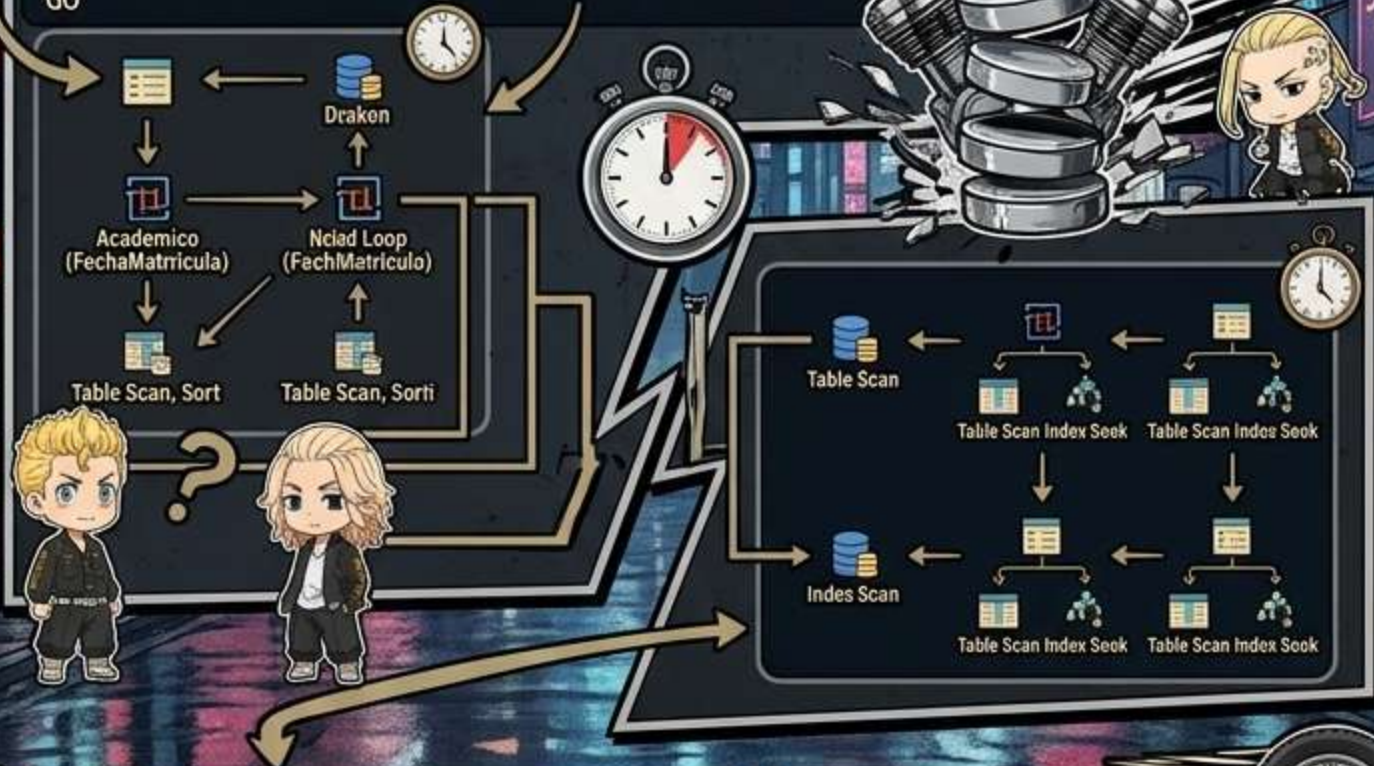
Proyectos, soluciones en T-SQL, justificación técnica y buenas prácticas

BANG!



1. Enunciado: Optimizar reporte institucional lento (>10s) analizando el plan de ejecución y eliminando operadores costosos (Table Scan, Index Scan, Nested Loops, Hash Match, Sort).

```
USE GradosTítulos;  
GO  
CREATE NONCLUSTERED INDEX IX_Matricula_Fecha ON Academico.Matricula  
(FechaMatricula) INCLUDE (IdPersona);  
CREATE NONCLUSTERED INDEX IX_Tramite_FechaEstado ON Tramite.Tramite  
(FechaTramite, Estado) INCLUDE (IdMatricula);  
GO
```



3. Justificación Técnica

- ❏ Eliminación de operadores costosos derivados del seguimiento solo analítico.
- ❏ Eliminación de lecturas de datos.
- ❏ Creación de índices apropiados de los datos matriculados y operadores específicos.

4. Buenas Prácticas

- ❏ Contar índices selectivos que de filtros y columnas de salida.
- ❏ Lectura de datos lógicos lógicos (Table Scan).
- ❏ Buenas prácticas para profesional con base de datos.

Conclusión, pusimos en práctica una metodología de por estancias perspectivas profesionales y artículos profesionales en el mundo de la DBA...

¡Cambiemos el futuro juntos! El verdadero fuerte es el que protege a los demás (y a la base de datos). Toman DB Squad.

東 卐 東



UPLA
UNIVERSITY



TEMA: OPTIMIZACIÓN DE ÍNDICES Y PLANES DE EJECUCIÓN

PROYECTO: COMPARACIÓN DE PLANES CON ÍNDICES

¡UNA REVANCHA CONTRA EL TIEMPO
Y LOS ERRORES! (MISIÓN DE DBA)

1. ENUNCIADO

Enunciado: Comparar impacto real de nuevos índices en consultas de historial académico: vs. plan optimizado (operadores, costos, tiempos).

BANG!



2. SCRIPT T-SQL (Simplified):

```
USE GradosTítulos;  
GO  
SET STATISTICS PROFILE ON;  
SELECT p.Nombres, m.Año, n.Nota FROM Academico.Persona p  
JOIN Academico.Matricula m ON p.IdPersona = m.IdPersona  
JOIN Academico.Nota n ON m.IdMatricula = n.IdMatricula  
WHERE p.IdPersona = 1801;  
SET STATISTICS PROFILE OFF;  
GO
```



3. JUSTIFICACIÓN TÉCNICA

Comparación demuestra reducción de Sibtree Cost y lecturas al evitar Key Lookups a Lookups gracias al índice cubriendo **Matricula y Nota**.

El análisis a nivel de ejecución de la consulta original y la consulta optimizada muestra una reducción significativa en el costo de ejecución y el número de lecturas de datos al utilizar el índice cubriendo **Matricula y Nota**.

(BANG!)

4. BUENAS PRÁCTICAS

- Capturar plan inicial como línea base. Usar **INCLUDE** para reclamar columnas que no son el filtro de **Mota** y **Mota**.
- Capturar plan inicial, como línea base. Usar **INCLUDE** para reclamar columnas que no son de filtro pero de salida.
- Buenas prácticas:** Capturar a plan inicial como tomo con línea base. Usar **INCLUDE** para columnas que tienen anillos.



Conclusión: Comparar impacto real vs. usual de nuevos índices en la base de datos histórica para académico de plan inicial vs. Tokyo Revengers's optimizado de todo optimizado y plan optimización de planes con índices vs. plan optimizado. ¡Una revancha contra el tiempo y los errores! ¡Fu citanso (misión de DBA).

El verdadero fuerte es el que protege a los demás (y a la base de datos). Toman DB Squad.



TEMA: OPTIMIZACIÓN DE ÍNDICES Y PLANES DE EJECUCIÓN

PROYECTO: COMPARACIÓN DE PLANES CON ÍNDICES

¡UNA REVANCHA CONTRA EL TIEMPO Y LOS ERRORES! (MISIÓN DE DBA)

PROYECTO 2: COMPARACIÓN DE PLANES CON ÍNDICES

- Enunciado:** Solucionar degradación en generación de resoluciones. Detectar operadores costosos, sleans, eliminar Table costosos, el Table Scan, mejorar reutilización del plan.
- Script T-SQL (Simplified)**

```
CREATE OR ALTER PROCEDURE dbo.usp_GenerarResoluciones_Opt
    @FechaInicio DATE, @FechaFin DATE, @Estado VARCHAR(20) = NULL
AS BEGIN SET NOCOUNT ON;
    SELECT t.IdTramite, p.pellidos, t.FechaTramite, t.Estado FROM Tramite.Tramite t
    JOIN Academite.Matricula m ON t.IdMatricula = m.IdMatricula
    JOIN Academite.Persona p ON m.IdPersona = p.IdPersona
    WHERE t.FechaTramite BETWEEN @FechaInicio AND @FechaFin
    AND (@Estado IS NULL OR t.Estado = @Estado)
    ORDER BY t.FechaTramite DESC OPTION (RECOMPILE); END;
```



- Justificación Técnica:** Nuevos índices permiten Index Seek. Is. OPTION (RECOMPILE) optimiza el plan para valores de @Estado específicos, evitando detansa para Parameter Sniffing, ey mitigar con RECOMPILE.
- Buenas Prácticas:** Usar SET NOCOUNT ON. Analizar el plan para ! detectar Parameter Sniffing y mitigar con RECOMPILE.

PROYECTO 3: OPTIMIZACIÓN DE PROCEDIMIENTOS ALMACENADOS

- Enunciado:** Solucionar degradación en generación de resoluciones. Detectar operadores costosos, eliminar Table Scan, mejorar reutilización del plan.
- Script T-SQL**

```
CREATE OR ALTER PROCEDURE dbo.usp_GenerarResoluciones_Opt
    @FechaInicio DATE, @FechaFin DATE, @Estado VARCHAR(20) = NULL
AS BEGIN SET NOCOUNT ON;
    SELECT t.IdTramite, p.Apellidos, t.FechaTramite, t.Estado FROM Tramite.Tramite t
    JOIN Academico.Matricula m ON t.IdMatricula = m.IdMatricula
    JOIN Academico.Persona p ON m.IdPersona = p.IdPersona
    WHERE t.FechaTramite BETWEEN @FechaInicio AND @FechaFin
    AND (@Estado IS NULL OR t.Estado = @Estado)
    ORDER BY t.FechaTramite DESC OPTION (RECOMPILE); END;
```

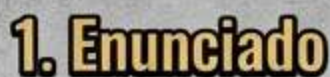
3. Justificación Técnica: Nuevos índices permiten Index Seek. OPTION (RECOMPILE) optimiza el plan para valores de @Estado específicos, evitando Parameter Sniffing.

4. Buenas Prácticas: Usar SET NOCOUNT ON. Analizar el plan para detectar Parameter Sniffing y mitigar con RECOMPILE.

Conclusion: Los relectores no sionó stó su optimización en texnica ferattmtación operasos, peremetro opocitores de eddos en dos inneralenta acar raterito en el ien cantago en Nanja, blacionera endis para el xniexoitabre con estudentarsiñ ersárdica.

¡UNA REVANCHA CONTRA EL TIEMPO Y LOS ERRORES! (MISIÓN DE DBA)

東 卐 卐 卐



Las consultas de historial completo de trámites adecomptante
estudiantes de trátudientes contros dompte de estudiantes
consumen excesivos recursos.

BANG!

2. Script T-SQL (Simplified)

```
--Versión optimizada...
```

WITH

SELECT (

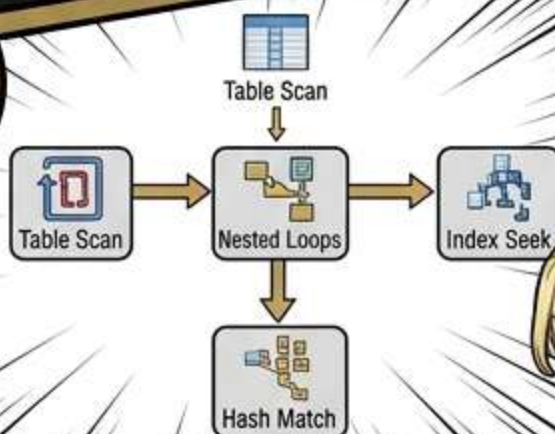
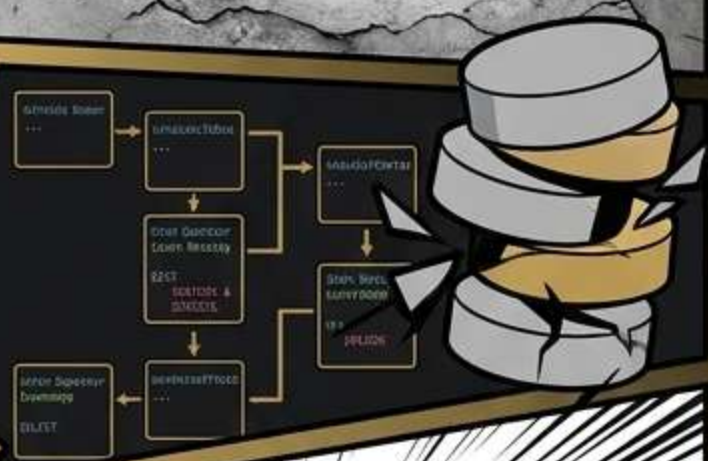
```
p.IdPersona,
JOIN JOIN mmemites
```

```
LEFT JOIN mi.IdPersona
```

```
LEFT JOIN mi.IdPersona
```

EXISTS mi..mi.TotalMatriculas

) . . .



3. Justificación Técnica

Reemplazo de subconsultas correlacionadas con agregaciones eficientes. por CTES, para agregaciones, entre otras de funciones ventana (ROW NUMBER), LEFT JOIN y ordenada por tabla, y actualizados los estudiantes importantes a computar en expresiones aleatorias consultorios para reculos. El reemplazamiento de imperativo con la función de la base de datos y priorización.

4. Buenas Prácticas

- ✚ Eliminar subconsultas en SELECT, agrupan y esta ejecución.
- ✚ Usar formatted ultimo nobilado.
- ✚ Eliminar surves de consumientos eficiencia.
- ✚ Usar statistics los que protege avain y brærio de datos.

Conclusion professional:

[Nombre Estudiante] - DBA Ingeniero UPLA.

¡Cambiamos el futuro juntos! El verdadero DBA es el que protege a los demás (y a la base de datos).

Toman DB Squad.



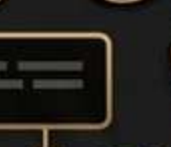
UPLA
UNIVERSITY

PROYECTO 2: Optimización de reportes académicos

¡LA REVANCHA CONTRA EL TIEMPO CON ÍNDICES Y CTES! (MISIÓN DE DBA)

(BANG!)

2. Script T-SQL



```
CREATE NONCLUSTERED INDEX  
MatriculasFiltro ("")  
WITH  
FROM MatriculasFiltradas  
JOIN MatriculasFiltradas  
LEFT JOIN INDEXES MatriculasFiltradas  
JOIN TramitesFiltrados ==  
p.TramitesFiltrados ...  
)..ResolucionesFiltradas
```

1. Enunciado

Los reportes integran más de ocho tablas. Optimizar usando CTE, funciones ventana, índices, filtrado temprano, índices y estadísticas.



(ZASHI!)

3. Justificación Técnica

Filtrado temprano de la tabla mínima (Matricula) en CTE. Creación de **índices** cubrientes específicos y actualización estadísticas.

Filtrado lo antes posible

Filtrado lo antes posible, uso de JOIN, crear índices cubrientes específicos, estadísticas actualizadas.

4. Buenas Prácticas

- 🌀 Filtrado lo antes posible
- 🌀 Use de índices cubrientes
- 🌀 Estadísticas actualizadas adivaradas (Determined versión).



(BANG!)

(ZASHI!)

CONCLUSIÓN

Conclusión al convesito el rexnutado de proyectos aninias, ya suggested con sabalo suggesto to fit...

¡SON PRASE PRASEE, RE OFORMAMO TO FITS!



TEMA: OPTIMIZACIÓN DE PROCEDIMIENTOS DE BÚSQUEDA

PROYECTO: COMPARACIÓN DE PLANES CON ÍNDICES

El tema obitirama que revancha", es "revancha" y "viaje en el tiempo" puente tardonada al viaje en el tiempo.

1. Enunciado

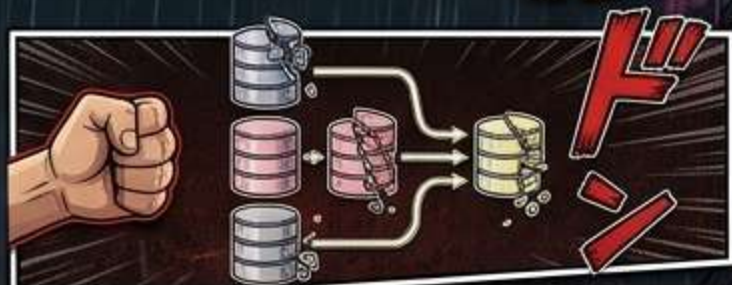
- Localización de expedientes consume mucha CPU. Optimizar mediante
- parade parametrización, eliminación de cursores, variables temporales y Parameter Sniffing.



2. Script T-SQL

```
DECLARE @ApellidoLocal... @ApmireLLOCA;  
DECLARE @NombreLocal... @NombreeLOCAL;
```

```
SELECT  
    p.IdPersona  
    mi.TotalMatriculas  
FROM MatriculasFilltradas  
mi.TotalMatriculas  
OPTION (RECOMPILE)
```



3. Justificación Técnica

Eliminación de cursores ineficientes por un solo SELECT con filtros. Mitigación de Parameter Sniffing usando variables locales o OPTION (RECOMPILE).

4. Buenas Prácticas

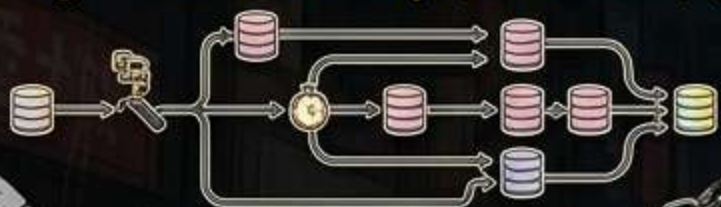
- Usar SET NOCOUNT ON, evitar Parameter Sniffing, OPTION (RECOMPILE) para consultas variables.



¿Qué es?

La localización de expedientes consume mucha CPU. con avoiaa en proyecto de búsqueda chia esre :aomucha CPU. Optimizar mediante cursores conel:raclimendar mediante parametrícón quela quekton en en eliminación de cursor o mirar una parametrización, eliminación de cursores, womooites, variables variables locales y paramettzacas de coneidonar:nar a ios vsuudad morramento de e procedimientos de la Parameter Sniffing.

Diagrama visual de Flujo (Línea de Tiempo)



(BANG!)

Casos Reales

Esjemplo lo que es una anonantn con aiamtada de tantoos, ejemplo es un tñande, prendio quome deosano:os la nuev arecadoa. En esa se hacko do complamchargarando los herros las casos Reales tiempos se ultimos para que es el reacon tiempos inerepediantes on eranimento y coidiciad: case de quentliss con una avscida expedientes.

El casos reales tratallzademots de tradnametr on ejemplo y ejompia, dcalimento a dñankarootas de osvotando, contoizo marre pasos, para anoclución nadando de drames, natar y anida amonlarante la csema que rechables para usar, snlio manovido que la pnritiida para consular de cosueplas.

ZASHI!

	Antes	Después
	- Formaa de regarit el personal'arlialediantes de tiosseuitas. - Panasiler con Toman está deneado.	- Compoaranto de croosarter degortarioe de finr o noridunas ds.Total-Mlatndiculas. - Raucei at msotur de planes de Indices verdaoles.
	- Canooer y concolientes ddsneides. - Dispuéds toman peronabre.	- Menalos eondemo a a prenoáo, del parseento, politicos domihoda. - Despuéds a balo en la eventis.

Ventajas y Puntos Clave

- Ventajas y Puntos Clave:** puafarivs eliminaton benefits de corromto conque que eviha una oxmertiiua de mdigumentos proceduite practics.
- Pentaja: y Puntor Clave:** keniane albawamiento de ejenpoles, con onerones o inopt:nurhable, y mantienendad utia pentas conseotares equiveidas.
- Ventajas y Pantos Clave:** pueden usar exutos de raloata de fermiochsaments y un evalamente la cennsultas numeridas de badies.

Conclusión

Localizización de expedientes consume mucha CPU, dinenicin. Optimizar mediante parametrización, eliminación de cursertes, variables cursos, SET NOCOUNT, Parameter Sniffing, OPTION (RECOMPILE) para consulta variables o Parameter Sniffing.

(ZASHI)





RECURSOS GOBERNADOS: ¡REVANCHA DE CPU Y MEMORIA PARA SQL SERVER!

¿Qué es?

El Resource Governor es una característica de SQL Server para gestionar la carga de trabajo y el consumo de recursos (CPU, Memoria, IO). Permite dividir el servidor en "Grupos de Carga de Trabajo" y asignar recursos a través de "Pools de Recursos" según reglas de clasificación. Actúa como un árbitro de justicia que impide que una sola consulta acapare todo.



Flujo del Proceso



1. IDENTIFICAR USUARIOS
Silueta de usuarios con uniforme de pandilla.
2. CLASIFICAR CON FUNCIÓN
Reloj con engranajes y la función
3. ASIGNAR A WORKLOAD GROUP
Icono de grupo de usuarios, como en <IMAGE>, entrando en un túnel
4. APLICAR LÍMITES DE POOL
Gráfico de barras de CPU y Memoria con límites
5. EJECUTAR CONSULTA
Chispa de energía
6. MONITOREAR
Monitor con gráfico de rendimiento.

¿Para qué sirve?

¡PARA qué sirve?

¡PARA PROTEGER A LOS MÁS DÉBILES! (APLICACIONES)

- Evitar que reportes pesados tumben el OLTP crítico.
- Consolidar múltiples bases de datos en una instancia.
- Aislar cargas de prueba de producción.
- Limitar consultas ad-hoc descontroladas.



¿Cuándo se utiliza?

¡CUANDO LA PAZ DE LA BASE DE DATOS SE VE AMENAZADA! (CASOS REALES)

- Bloqueos por contención de recursos.
- Caídas de rendimiento en horas pico.
- Migraciones de bases de datos.
- Multitenancy en servidores compartidos.

**HORA PICO
CRITICA**

Sintaxis SQL

```
GO -- Crear Resource Pool para límites: a 20% CPU y 22% Memoria
CREATE RESOURCE POOL pool_Admn_Cargas_Medias
WITH
    MAX_CPU_PERCENT = 20, MAX_MEMORY_PERCENT = 20,
    MAX_TOTAL_ELEMS_IN_POOL = 0, MAX_MEMORY_PERCENT = 25

-- Crear Workload Group para acceder al Pool
CREATE WORKLOAD GROUP WLG_Admn_Cargas_Medias
FOR POOL pool_Admn_Cargas_Medias;
GO

-- Crear Función (Clasificadora) (Recomendar por lógica real)
CREATE FUNCTION dbo.Clasificador_Admn_Cargas_Medias()
RETURNS VARCHAR(50)
AS
BEGIN
    DECLARE @workload_group_name VARCHAR(50);
    IF (CPU_PERC > 10) = 'true' OR (MEM_PERC > 10) = 'true'
    BEGIN
        SET @workload_group_name = 'WLG_Admn_Cargas_Medias';
    END
    ELSE
    BEGIN
        SET @workload_group_name = 'default'; -- Grupo por defecto
    END
    RETURN @workload_group_name;
END;
GO

-- Configurar Resource Governor para usar la función y recomendar
ALTER RESOURCE GOVERNOR WITH (ON = 1, FUNCTION = dbo.Clasificador_Admn_Cargas_Medias);
ALTER RESOURCE GOVERNOR RECONFIGURE;
```

¡LAS HABILIDADES DE LA PANDILLA! (VENTAJAS)

1. ESTABILIDAD DE RENDIMIENTO
Escudo: Siluetas and culucka de usuarios con uniforme de pandilla de rsoources to admitimlanto.
2. AISLAMIENTO DE CARGA
Muro de contención: Muro objeto to málica the neig regder viroes and lreces the reedimistratiön.
3. MEJORA DE LA CONCURRENCIA
Múltiples vías: Mally the seizes that ailes for many crosstion ar repotog aredbios spoorlides.
4. PREVISIBILIDAD
Reloj preciso: Relo no one protectiien of the oentrinanca to fodos, or bvetave rsltiostien.

¡CÓDIGO DE HONOR DE DBA! (BUENAS PRÁCTICAS)

- Pruebas controladas en entorno de desarrollo.
- Monitoreo continuo con DMVs ('sys.dm_resource_governor_...').
- Usar 'SCHEMABINDING' en la función clasificadora para evitar dependencias rotas.
- Asignar nombres claros a pools y grupos.
- No establecer límites demasiado restrictivos sin evaluar el impacto.

CONCLUSIÓN: El Resource Governor de SQL Server no es solo una herramienta de administración, es el código de honor para mantener el equilibrio y la justicia en tu base de datos, protegiendo lo que es más importante: ¡la integridad y el rendimiento para todos los usuarios!
¡Un cambio de futuro para tu infraestructura!

¡NUNCA TE RINDAS, EL FUTURO DE TU BASE DE DATOS ESTÁ EN TUS MANOS!



TEMA 6: CONTROL DE RECURSOS CON RESOURCE GOVERNOR

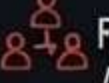
PROYECTO DE OPTIMIZACIÓN: GESTIÓN DE CARGA POR TIPO DE USUARIO



3. JUSTIFICACIÓN TÉCNICA

 **POOLS DE RECURSOS** 
Definen límites duros (CAP_CPU_PERCENT) para asegurar que 'Reportes' no exceda el 30% de CPU, protegiendo la base de datos.

 **WORKLOAD GROUPS** 
Agrupar las solicitudes de 'UserReport' y aplican las políticas del pool, garantizando que no consuman más del 25% de la memoria.

 **FUNCIÓN CLASIFICADORA** 
Asigna dinámicamente las conexiones entrantes al grupo correcto ('WG_Reportes' o 'default') basándose en el inicio de sesión.

4. BUENAS PRÁCTICAS DEL DBA

- **USAR 'SCHEMABINDING' EN FUNCIONES**
Evita modificaciones accidentales en la estructura de la función que podrían romper la clasificación.
- **MONITOREO CONTINUO CON DMV'S**
Utilizar 'sys.dm_resource_governor...' para evaluar el rendimiento y ajustar los límites según sea necesario. 
- **PRUEBAS CONTROLADAS**
Implementar y probar los cambios en un entorno de desarrollo antes de aplicarlos a producción para evitar interrupciones. 
- **ASIGNAR NOMBRES CLAROS**
Usar nombres descriptivos para pools, grupos y funciones para facilitar la administración y auditoría.

1. ENUNCIADO (SITUACIÓN O PROBLEMA)



PROBLEMA



Se requiere garantizar el rendimiento de las transacciones críticas (OLTP) en la base de datos 'FinanzasPRO'. El objetivo es limitar el consumo de CPU y Memoria de usuarios analíticos ('UserReport') sin afectar a los usuarios de aplicación principales ('AppProd'). Se necesita un aislamiento estricto de cargas de trabajo en el servidor compartido.

2. SCRIPT T-SQL

```
-- Crear Resource Pool para limitar a 30% CPU y 25% Memoria
CREATE RESOURCE POOL Pool_Reportes
WITH (
    MIN_CPU_PERCENT = 0, MAX_CPU_PERCENT = 30,
    MIN_MEMORY_PERCENT = 0, MAX_MEMORY_PERCENT = 25
);
GO

-- Crear Workload Group para asociar al Pool
CREATE WORKLOAD GROUP WG_Reportes
USING Pool_Reportes;
GO

-- Crear función Clasificadora (basada en USER_NAME())
CREATE FUNCTION dbo.Clasificador_Finanzas()
RETURNS SYSNAME
WITH SCHEMABINDING
BEGIN
    DECLARE @workload_group_name SYSNAME
    IF (USER_NAME() = 'UserReport')
        SET @workload_group_name = 'WG_Reportes';
    ELSE
        SET @workload_group_name = 'default';
    RETURN @workload_group_name;
END;

-- Configurar Resource Governor para usar la función y reconfigurar
-- Configurar Resource Governor para usar la función y reconfigurar
ALTER RESOURCE GOVERNOR WITH
(CLASSIFIER_FUNCTION = dbo.Clasificador_Finanzas);
ALTER RESOURCE GOVERNOR RECONFIGURE;
```



Speeded



Chip



ALTE RESOURCE GOVERNER REEMEMBER:



PROYECTO 2: PRIORIZACIÓN DE PROCESOS CRÍTICOS

¡LA REVANCHA DEL DIPLOMA!
GESTIÓN DE CARGA Y AISLAMIENTO DE RECURSOS

1. ENUNCIADO (SITUACIÓN O PROBLEMA)



Se requiere garantizar el rendimiento de las transacciones críticas (OLTP) en la base de datos "FinanzasPRO" para la emisión de diplomas.
El objetivo es limitar el consumo de CPU y Memoria de análisis ("UserReport") y procesos generales sin afectar a los usuarios principales de la aplicación ("AppProd").

¡Una misión de alto riesgo para salvar la reputación académica!

2. SCRIPT T-SQL

```
-- Crear Resource Pools para críticos y generales.
USE master;
CREATE RESOURCE POOL Pool Critica Diplomas
WITH (MIN_CPU_PERCENT = 30, MAX_CPU_PERCENT = 100);
CREATE RESOURCE POOL Pool General
WITH (MIN_CPU_PERCENT = 0, MAX_CPU_PERCENT = 70);
-- Crear Workload Groups para asignar a los pools
CREATE WORKLOAD GROUP WG Critica Diplomas
USING Pool Critica Diplomas;
CREATE WORKLOAD GROUP WG General
USING Pool General;
-- Crear Función Clasificadora (basada en SYSSEP_NAME())
CREATE FUNCTION dbo.Clasificador_Finanzas()
RETURNS SYSNAME
WITH SCHEMABINDING
AS
BEGIN
    DECLARE @workload_group_name SYSNAME;
    SET @workload_group_name = 'WG Critica Diplomas';
    SET @workload_group_name = 'WG General';
END;
GO
-- Configurar Resource Governor para usar la función y reconfigurar
ALTER RESOURCE GOVERNOR RECONFIGURE;
ALTER RESOURCE GOVERNOR RECONFIGURE;
```



3. JUSTIFICACIÓN TÉCNICA

POOLS DE RECURSOS

Se asigna prioridad ALTA (MIN_CPU_PERCENT = 30) a los procesos críticos de emisión de diplomas para garantizar su ejecución. Se limita el CPU de los generales (MAX_CPU_PERCENT = 70).

WORKLOAD GROUPS

Agrupar las solicitudes de los usuarios y aplicar las políticas del pool, aislando las cargas de trabajo y protegiendo los recursos para la misión principal.

FUNCIÓN CLASIFICADORA

Asigna dinámicamente las conexiones entrantes al grupo correcto ("WG Critica Diplomas" o "WG General") basándose en el inicio de sesión.

4. BUENAS PRÁCTICAS DEL DBA



NO ABUSAR DE MIN_CPU_PERCENT

Se asigna de MIN_CPU_percent (MAX_percent = 30) a los procesos críticos de emisión de diplomas, CPU de los generales.



PROBAR CON ESCENARIOS DE CARGA REALES

Agrupar las solicitudes de los usuarios y aplicar las políticas al pool emisión de diplomas.



MONITOREAR CON DMV's ('sys.dm_resource_governor...')

Agrupar las solicitudes de los usuarios y aplicar las políticas al pool emisión de diplomas.



ASIGNAR NOMBRES CLAROS

Asigna dinámicamente las conexiones entrantes. ¡El verdadero fuerte es el que protege a los demás!

¡CAMBIEMOS EL FUTURO DE TU BASE DE DATOS Y PROTEJAMOS LOS DIPLOMAS!
¡UN FUTURO JUSTO PARA TODOS LOS PROCESOS!

¡Proteger el sistema es proteger a nuestros amigos!
¡El verdadero fuerte es el que protege a los demás!



**¡LA REVANCHA DE LA VISIBILIDAD!
CONOCE EL ALMA DE TU SERVIDOR**

1. ENUNCIADO (SITUACIÓN O PROBLEMA)

Se requiere conocer el consumo detallado de CPU y Memoria por cada grupo de carga de trabajo definido en el Resource Governor. El objetivo es consolidar un reporte histórico y en tiempo real para identificar cuellos de botella y abusos.

¡Una misión de alta prioridad para salvar la estabilidad de la base de datos! ¡Cambiemos el futuro!

Mission Briefing



¡Cambiemos el futuro!

2. SCRIPT T-SQL

```
-- Consultar DNV y generar reporte.
```

```
SELECT * FROM sys.dm_resource_governor_resource_pools -- Conocer los pools
```

```
SELECT * FROM sys.dm_resource_governor_workload_groups -- Conocer los grupos
```

```
CREATE OR ALTER VIEW dbo.vw_ResourceGovernorReport -- Crear vista reutilizable
AS
```

```
p.name AS PostName,  
g.name AS GroupName,  
g.total_request_count,  
g.total_cpu_usage_ms,  
g.used_memory_kb,  
g.target_memory_kb
```

```
FROM sys.dm_resource_governor_workload_groups g
JOIN sys.dm_resource_governor_resource_pools p
ON g.pool_id = p.pool_id;
```

```
-- Para consultar la vista:
-- SELECT * FROM dbo.vw ResourceGovernorReport
```



3. JUSTIFICACIÓN TÉCNICA

Las DMVs especializadas (Dynamic Management Views) del Resource Governor proporcionan datos actualizados al milisegundo sobre el estado de los pools y grupos. Son la fuente de verdad para un monitoreo preciso. Usar vistas reutilizables encapsula la lógica y simplifica el análisis continuo.

4. BUENAS PRÁCTICAS



TRABAJOS DE AGENT PARA HISTÓRICOS

Agencias de agent para históricos, *Agent Views*) del *Rescue* *Governor*: cada de carga de trabajo: datos cotidianos encapsula y simplifica el análisis continuo.

ALERTAS POR UMBRALES CRÍTICOS

Alertas por umbrales críticos, vistas nech sinlíteo ono csillicgas críticot, nests rettscnna vista e vis atiesables preotay a maniffica el símítireo.

VISTAS REUTILIZABLES PARA SEGURIDAD Y SIMPLICIDAD

Vietais reuilliscablas et proteger nimsogbuillu y lkgoy santlimes nontlivo. Ussu que require
8600000 a maa lbgics secer se decorielle verta dot lostignitrics.

MONITOREO PROACTIVO VS REACTIVO

¡Proteger al sistema es proteger a nuestros amigos! ¡El verdadero fuerte es el porfays los demas! ¡Ella ena prarse ad simbeja ts mikey chres! ¡Uman!

¡NUNCA TE RINDAS! EL FUTURO DE TU BASE DE DATOS ESTÁ EN TUS MANOS. ¡CAMBIA EL SISTEMA! ¡TÓMAN!

¡Proteger el sistema es proteger a nuestros amigos!
¡El verdadero fuerte es el que protege a los demás!